

Building and Verifying a Multi-Agent LLM Pipeline

A Case Study in 74 Defects

Arya Pathare

August 2026

Repository: github.com/AryaPathare/ai-investment-agent — v1.0.0 (48f9c08), MIT licensed.
The 95-entry development log analysed here ships with it. Cloning the repository gets `main`;
`git checkout v1.0.0` to see what this paper describes.

Contents

1. Summary	3
2. What the system does	4
3. Architecture, and the rules behind it	6
4. How it was verified	7
5. What went wrong, and what kind of wrong it was	10
6. What is still wrong	13
7. Limits of this evidence	15
Appendix A --- how every entry was sorted	15

THE SYSTEM

5	8	2-4	6-7
agents, on a state machine	questions, plus a ninth only on a contradiction	minutes for a full run	runs a day on a free tier

THE WORK, UP TO V1.0.0

16	95	74	816
sessions	log entries, written as the work happened	defects pulled out and sorted	tests passing, 817 collected

WHAT THE LOG ACTUALLY SHOWS, UP TO V1.0.0

25 vs 11 Times a measuring instrument was wrong, against times the model misjudged something it was asked to judge.	3 of 74 Defects the 816-test suite found. Only one of those already existed rather than being introduced as the tests were written.
351 Passing tests on an agent whose model had never once been called. Every test stubbed the news client.	168 A profit margin, as reported by a data provider. The maximum possible value of a profit margin is 1.0.

The log figures are counted rather than estimated, and Appendix A lists the sorting behind them. The first band is measured from real runs.

1. Summary

I built a five-agent language-model pipeline that researches public companies for someone who has never invested. The person answers eight questions. The system finds themes in current news for the sectors they named, finds listed companies genuinely exposed to those themes, argues against each one, and writes up to three of them in plain language. Or it recommends nothing and says why, which is a designed outcome and not a failure. It runs on a state machine that saves its progress, so an interrupted run picks up where it stopped. The test suite reports 816 passed and 1 skipped at v1.0.0, CI runs on two platforms, and the repository is public.

Alongside it I kept a log, written as the work happened rather than assembled afterwards. It records every significant defect: what broke, what I first thought had broken, and the measurement that settled it. Over the sixteen sessions up to v1.0.0 it reached 95 entries. This paper sorts all of them. I pulled out 74 separate defects and put each into one of eight groups. The unit is a defect and not an entry, because a third of the entries record no defect at all and several record more than one.

The result is not the one I expected. I built the architecture to guard against a language model being wrong, and that guard mostly held. The biggest group of defects is a broken instrument — the test, count or check that was supposed to tell me whether something worked, and was itself wrong. An evaluation that divided by the wrong thing. A set of test cases that scored full marks because every one of them repeated a word, so simple word-matching and real judgment looked the same. A check that read the words used to describe a company instead of what the company does, and let an oil major through on a profile that excluded fossil fuels. A change measured with nothing to compare it against, which looked like a success on every number it produced and was a regression.

Three more findings follow. All eleven judgment failures have one shape: a prompt that said what I wanted without saying how the thing works, so the model's reading was reasonable and wrong. The test suite's real job turned out to be holding things steady after unrelated changes rather than finding new problems. And writing that does not match behaviour is rarely the cause of a defect and often the reason it stays hidden: 5 defects as a direct cause, 16 once indirect ones are counted, the widest gap in the chart.

Scope. One system, one model, one provider, and one author sorting his own notes after the fact, on a free API tier. I make no claim that these proportions hold anywhere else; Section 7 sets out the limits. One caveat belongs here rather than in a footnote. The first version of this paper's main table was wrong in six of its eight rows, because I added up my own sorting from memory instead of counting it. That error and two more like it are in Section 5.4. A paper arguing that measurement fails more often than the thing being measured cannot present its own numbers as if they arrived clean.

2. What the system does

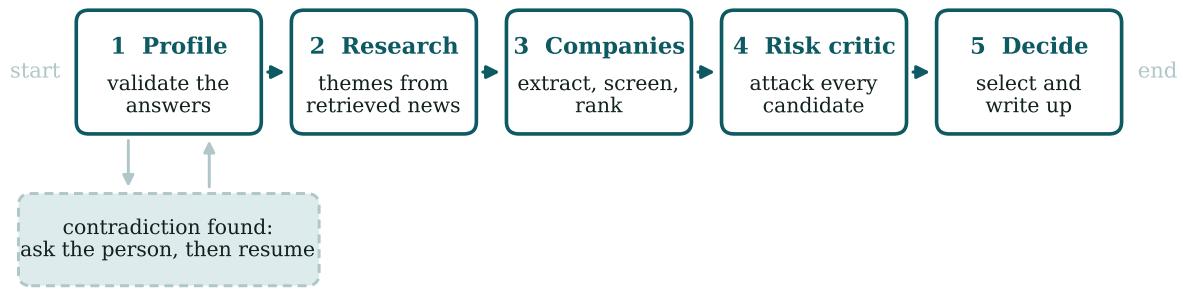


Figure 1: The five stages. Routing, retries and the pause for clarification are handled by the graph, not by a supervising model. Any stage may also return nothing, which ends the run with a stated reason rather than a blank screen.

At v1.0.0 the nine questions were age, experience, risk tolerance, amount, currency, investment window, holding period, sectors of interest, and restrictions. There are eight now: the two timeframe questions turned out to be one question asked twice, since the command line asked when the money was needed back while the agent's prompt read the answer as when the person planned to buy. One further prompt appears only sometimes: when the answers contradict each other — a sector named alongside a restriction that forbids it — the run stops, asks, and carries on from where it paused. Everything is saved as it goes, and resuming does not redo finished stages, which on a free daily budget is the difference between losing a minute and losing the day.

Figure 2 is a real run, replayed from a recording that ships with the code. Two of its three companies are cut for space. The full output is one command for any reader and needs no key, no network and no quota.

```
[ profile: age 56, beginner, low risk, USD 1,000 over 5 months,
  interested in Technology and Utilities, will not hold: no ]

=====
3 COMPANIES WORTH A LOOK
=====

1. NVIDIA Corporation (NVDA)
  Came up because of: AI Chip Capacity Crunch
  We argued against this one and it held up.

  Why it might be worth a look
    NVIDIA sells GPUs that power AI workloads. Its revenue grows as AI chip
    demand rises. The company is a good fit for a low-risk investor holding
    for ten months.

  What would mean the idea has stopped working
    - Microsoft secures majority of TSMC capacity for its Maia 300 chip.
      Check: "Microsoft's Maia 300 Chip Targets NVIDIA's AI Dominance"
        finance.yahoo.com, 16 Aug 2026
    - revenue growth turns negative.
      Check: the company's reported revenue growth
    - debt-to-equity rises above 1.0.
      Check: the company's reported debt-to-equity

[ 2. Samsung Electronics and 3. SMIC elided ]
```

```

ALSO CONSIDERED, BUT NOT RECOMMENDED
Alibaba Group Holding Limited (BABA) - Ranked just outside the top three.

WHAT TO DO NEXT
NVDA      USD 217.09 per share                      (as of 28 Aug 2026)
          Your USD 1,000 would buy about 4 shares.
SMSNN.MX  MXN 81,745.90 per share (about USD 4,799.41)
          One share costs more than your USD 1,000.
0981.HK   HKD 71.30 per share (about USD 9.09)
          Your USD 1,000 would buy about 109 shares.

Look at this again on 28 Nov 2026, about three months from now - roughly one
set of results. What to check is listed under each company above.

=====
This is research, not advice. It does not tell you what to buy, or how much.
=====

```

Figure 2. `python -m cli --demo`. Any reader can reproduce this in one command. It carries two defects found after it was recorded, both left in place. The profile says five months while the recommendation below reasons about ten, which Section 4 explains. And the line reading “will not hold: no” is the second: people answer that question with “none” or “no”, and a restriction is honoured when searches are built, so the word was carried in as an area not to research. It is now discarded in the model rather than in the prompt wording, which had already been ignored twice.

Four decisions are easier to see there than to describe. **Every condition says what would check it.** One points at a news article by headline, publisher and date; two name a number the company reports. A condition backed by neither is thrown away before the write-up is built, and the ones thrown away are counted. Seeing the same generic check repeated down the page is how a reader notices the write-up is thin, so the repetition stays rather than being tidied away. **The second line under “what to do next” is the honest one.** One share of the Mexican listing costs more than the person’s whole amount. Share counts round down, never up, so the system never tells someone they can afford a share they cannot, and a count of zero is written as a sentence rather than skipped. **There is a review date and no sell date.** Nothing here predicts a price, so a sell date would be the only figure in the output backed by nothing. The conditions are the sell signal instead, tied to events rather than to a date. **Nothing says how much to invest.** There is no field for it anywhere in the system, for reasons Section 3 gives.

An empty result is a real answer at every stage.

```

=====
NOTHING IS BEING RECOMMENDED
=====

This is a real answer, not a failure. Everything ran, and nothing it found
was good enough to put in front of you.

Why
  No companies reached this stage: the research found no investable company
  genuinely exposed to any theme.

```

Figure 3. A renewable-energy run. Not reproducible from a fresh copy of the repository, because saved runs are not committed.

The banner is the loudest thing the system prints, and the run still exits successfully, because an error code would tell any surrounding script the run had failed and a blank screen would look like a crash. Renewable energy reaches this outcome more often than semiconductors do, and that is not a defect. Only about three in ten companies examined on that theme can be invested in at all, because the news is dominated by private and foreign firms.

3. Architecture, and the rules behind it

Six rules did most of the design work. Each is written here with the failure it prevents, because a principle without one is decoration.

Models judge; Python computes. Anything that can be calculated stays in code. Of the nine steps in the company stage, two need a model: spotting which companies an article names, and judging whether a company is really exposed to a theme or merely turned up in the same paragraph. Everything else is arithmetic over reported figures, and the model contributes exactly one input to a company's rank. The reason is not distrust in general. A model asked to score a company out of 100 returns 72 with nothing behind it: you cannot reproduce it, compare it between companies, or explain it. So the judgments it does make are deliberately coarse — direct, partial, or incidental. One thing left out of the screening rules is worth naming: heavy borrowing is not disqualifying on its own, because a blanket debt limit would quietly remove every bank, insurer and utility from a system meant to research banking themes.

The model returns a verdict, never the data. Used three times. The first agent originally received the person's profile as its own output format, so it rewrote their age, amount, sectors and restrictions on every call and could change any of them. It now returns a status, a reason, and a short list of fields a clarification is allowed to touch, and Python copies the rest from what the person actually typed. That change deleted a prompt rule. The prompt used to say “preserve the user's information”, which existed only to paper over the design flaw, and once the model could no longer reach those fields the rule was dead weight. **When a prompt rule is patching a design problem, fix the design.** The second agent's evidence type has no field for a web address or a headline, so inventing a source is impossible rather than discouraged. The third agent's output has no field for a ticker symbol: NVDA, NVDA.NE and NVD.DE are all real symbols for the same company, so a guessed one does not look wrong — it quietly returns a different company's finances.

Put rules in types, not comments. Company figures arrive in the currency of that company's accounts, including pence reported as hundredths of a pound without saying so, and only four ratios can be compared across companies. So comparable ratios and money amounts are separate types, and reaching for the wrong one means crossing a type boundary. I learned this from a failure: two data providers used one field name for two different things — what the accounts are in, and what the shares trade in — and a net income of 162 trillion won was labelled as dollars.

Every loop is bounded by code. The clarification cycle stops after a set number of attempts and records why it gave up. The same idea removed a whole component. The original plan had a supervising agent to route between stages, and everything in its job description is already

a feature of the graph: edges for routing, a policy for retries, an interrupt for clarification, schemas for validation. **A model routing between stages would be less reliable than an edge.** For the same reason there is one retry layer and not two, since the model client already retries on its own and stacking a second layer multiplies attempts against an API that is rate-limiting you.

Rejections are recorded, never dropped. “Three candidates from thirty mentions” is either good filtering or a broken lookup, and from outside those look the same. A summary records where every examined company went, and it caught three real defects on its first live run. The same reasoning applies to the filters in front of the risk critic, which report what they held back, because “no risks found” means one thing when every article reached the model and another when four of twelve never did.

Two decisions only work as absences. There is nowhere to put a position size. Splitting the money three ways is two lines of code, and it is the point where research turns into advice; a system with no field for it holds that line better than a prompt asking the model not to cross it. And the final agent calculates no new score — the company score and the risk verdict are carried through rather than combined, so that stage cannot disagree with the ones before it. Selection goes by risk verdict first, with the company score breaking ties inside each tier.

The risk critic works the same way: Python builds the searches and fetches the articles, the model reads them and returns risks, Python throws away any risk that cites nothing it was given, adds risks worked out from the company’s figures, and calculates the verdict. Those figure-based risks are never shown to the model, because a model handed a balance sheet and asked what could go wrong writes fluent sentences you cannot check against the numbers. They also give the agent a floor: in a quiet news week nothing comes back, and a critic that reports nothing reads to a person as reassurance. Every risk must point to an article or a named figure, and a check refuses to build one that points to neither.

That confirms a claim can be checked, which is necessary and not enough. Section 5 has the case where it was not.

4. How it was verified

Running the test suite at v1.0.0 reports 816 passed and 1 skipped in a few seconds, with no network and no API key. Those tests prove the code does what it says. The evaluations make real API calls against labelled cases and measure the model’s judgment, which varies between runs, so they are a measurement to compare across changes rather than a pass-or-fail gate. That difference is the project: every significant defect in sixteen sessions came from an evaluation, a live run, an audit or a person reading the output. The suite stayed green while all but three of them were present.

That is not a poor showing, because finding new problems is not what a test suite is for. Its job is holding something steady after you change something else, and it did that repeatedly. A test went red when a new type was added to the saved state, before that type had reached any agent. A test that walks the examples folder rejected a recorded run put there by mistake, catching the error in about four seconds. A rule comparing two profit margins, added late,

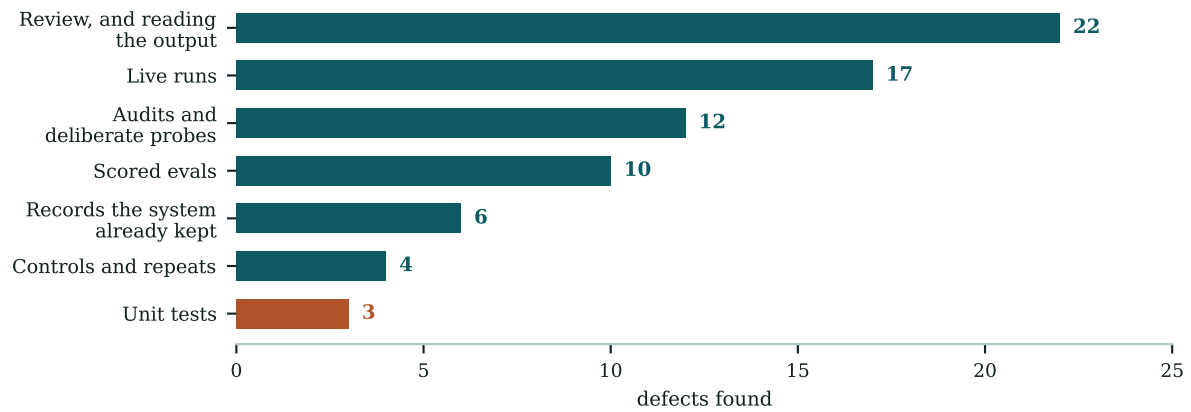


Figure 2: What found each of the 74 defects. The suite is in red. It found three, and only one of those was a defect that already existed rather than one introduced while the tests were being written.

went red on two of the project’s own test fixtures, which described a company whose margins could not both be true and had passed for eleven sessions because nothing had ever compared them.

Four instruments, in the order I found them, which is not the order of their worth.

Unit tests. Cheap, fast, and blind to everything the model does.

One evaluation per agent. These found most of the defects in the middle sessions. They also cost the most, because they chain: measuring the company agent runs the research agent first, and measuring the final agent runs the whole pipeline. Chaining has a second cost that took several sessions to see. The research stage fetches different articles every run, so an evaluation of a later agent is measuring it over different material each time. Three defects were blamed on that variation before one was traced further up: the same profile generates completely different searches between runs, and one set asked about processes and permits, which return articles about governments, while the other asked about manufacturing and contracts, which return articles about companies. “The results vary” was true and stopped one step short of the cause. The evaluation runners were themselves wrong 25 times, and for ten sessions not one of them had a single test.

One of them was worse than wrong. A case tagged regression asserted that the two timeframe fields could never contradict each other, which was true of what those fields were meant to mean and false of the question being asked. The recording in Figure 2 needs the money back in five months and holds for ten, and that evaluation required the agent to pass it. **A guard written against the intended meaning is no guard once the wording drifts.** Every other broken instrument here failed to notice something; that one protected it.

There is a limit no amount of care inside an evaluation gets past. **Every test profile was written by someone who knew what the field was for.** The first time a person outside the project used the system, they answered “which sectors interest you?” with “*whichever will make me the most money*” — not a contradiction, so it passed the agent built to catch contradictions, and the run died three stages later with six companies from three unrelated industries. Ten sessions of labelled evaluation had not produced anything like it, because real people answer the question they wish you had asked.

Replaying saved state. Every run is saved so a person can resume it. It turned out to be the best instrument in the project, because it is the only place a real run's middle state is kept. Replaying one agent over a finished run's saved inputs costs one stage instead of the whole chain, removes the article variation from measurements that have nothing to do with it, and compares against a recorded result rather than a remembered one. It checked a grading change against the exact eight articles that produced the grades I was complaining about. It checked a fix against the exact error text from the run that died. It rescued a run that failed at the last stage for the cost of one model call instead of twelve. And it fired a safety check that no live run had ever reached, at no quota cost at all. I found this at session 10 of 16, and it should have been the main instrument from session 2.

Comparison runs and repeats. One case in the first agent's set returns different answers on identical input, which puts about one point of noise on any single twelve-case score. Comparison runs came later still. The clearest case is a prompt change meant to give the company agent more articles to work with. Measured on its own it looked like a success: no hard failures, complete data, a candidate produced. Measured against an untouched run of the same profile in the same session it was worse on every axis, and the reason was visible in the first column. Told to attach every supporting article to a theme, the model merged five themes into three, and since companies are graded against themes, the number of themes matters more than the number of articles per theme. It was reverted. **A single number could not have shown this and would have been read as success.**

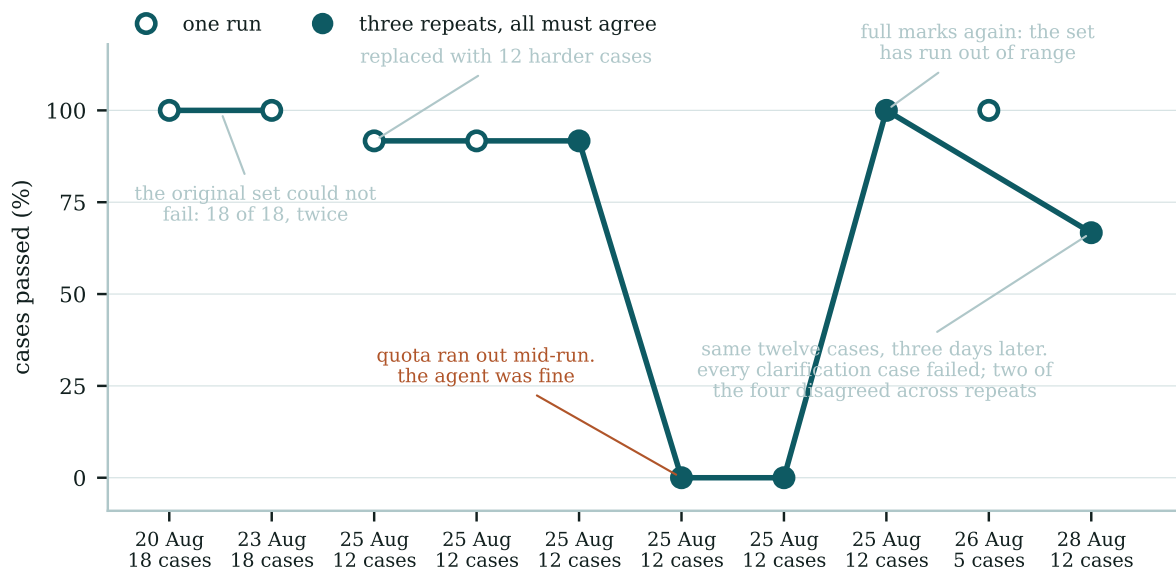


Figure 3: Every recorded run of the first agent's evaluation, in order. Hollow markers are a single run; filled markers repeat each case three times and require all three to agree. Lines join only runs that used the same set of cases.

Every run of that evaluation was kept, and plotting them in order shows more about the instrument than about the agent. It opened at full marks twice, which was the problem rather than the achievement: the cases were separable by word overlap, so the set could not fail. Replacing it with twelve harder cases made it discriminate again, and it lost that power within a week by scoring full marks once more. **The two runs at zero are the clearest thing on the chart.** Nothing was wrong with the agent. The daily token budget ran out partway through, every remaining case came back as a refusal, and the evaluation reported total failure. An

instrument that cannot tell a broken agent from an exhausted quota will report the same number for both. The final run sits at 66.7 per cent on exactly the same twelve cases three days later, with every clarification case failing. Two of those four failures are not wrong answers at all: the verdict was right, and the case failed because three repeats of identical input did not agree with each other.

What that ordering cost. Every single-shot score before session 10 has an error bar nobody measured: two results recorded as “11 out of 12, unchanged” on one day were not the same result, because the first failed one case and passed another and the second did the reverse. And every prompt change before session 14 was measured with nothing to compare it against. Neither of these makes the findings wrong. Both mean the earlier ones are weaker evidence than their numbers suggest.

Measuring under a quota. Six or seven full runs a day is most of the reason replaying saved state mattered so much, and it produced a finding of its own. The provider does not report the daily budget, so I once tested for remaining room with a deliberately tiny call and read its success as permission to start a 36-call run. Seventy-nine tokens were left. A one-token probe fits inside seventy-nine tokens, so the probe could not have failed, and its success told me nothing. What works instead is the refusal itself, which states the limit, the usage and the size of the rejected request exactly, and costs nothing. That is the same lesson as a test one session earlier, where deliberately breaking a safeguard left every test passing and showed the tests were pointed at nothing. **A test that cannot fail is not evidence**, and the only way to find out whether it can is to break the thing on purpose.

What may be substituted. Two of four exclusion reasons had never fired on a real run. I fired the one that stops a forbidden company reaching a person by taking a finished run’s real candidates and changing the investor’s restriction. I did not fire the other, which triggers when the critic finds a critical risk, because that would have meant writing the critical risk myself. A restriction is something the user types and can legitimately differ, so a run with a different one is a different run and not a fake. A critique is something the system produces, and replacing it gives you a unit test dressed up as a live run.

5. What went wrong, and what kind of wrong it was

5.1 How the sorting works

The unit is a defect, not a log entry. Several entries carry more than one: one records three separate ways a correct answer was lost on its way out of the model, another records seven findings from a single review. A third of entries record no defect at all — a design decision, a check that came back clean, an idea rejected on evidence — and counting those would inflate every group equally. Where a defect has two causes it gets a main one and a secondary one; the headline counts use main causes only, with both shown so the double-counting is visible. The counts come from a script reading the sorting table, not from adding them up by hand, for reasons Section 5.4 gives.

A. Envelope and transport. The right content, lost in its wrapper. A plain list where a labelled object was expected. A field set to nothing where the system required a list. In one case a

recovery routine written for exactly this problem was made unreachable by a fix I had applied eight hours earlier, because switching output methods changed which channel the error came back through.

B. Provider semantics. One field name meaning two things depending on who filled it in, or a value that cannot be true. A borrowing ratio reported as a percentage by one source and a plain ratio by another, exactly a hundred times apart. A profit margin of 168.

C. A rule met input its author never imagined. Removing duplicate headlines by similarity, against financial headlines that are near-identical and differ in exactly the word carrying the meaning. Matching company names word by word, against “AMD” and “Advanced Micro Devices, Inc.”, which share no word. A filter meant to remove company announcements that removed “*First Solar Reports Disappointing Second-Quarter Financial Results, Shares Plunge*”.

D. Downstream starvation. A stage fed the leftovers of a filter one stage earlier that was never meant to feed it.

E. Silent state and plumbing. Nothing raised an error and something was quietly wrong later. Types missing from a saved-state list, which come back as plain dictionaries and fail somewhere else entirely.

F. A broken instrument. The measurement was wrong, so the finding was wrong.

G. Model judgment. The model was wrong about something it was asked to judge.

H. Writing that does not match behaviour. A document, comment or note claiming something the code does not do.

5.2 What the counts show

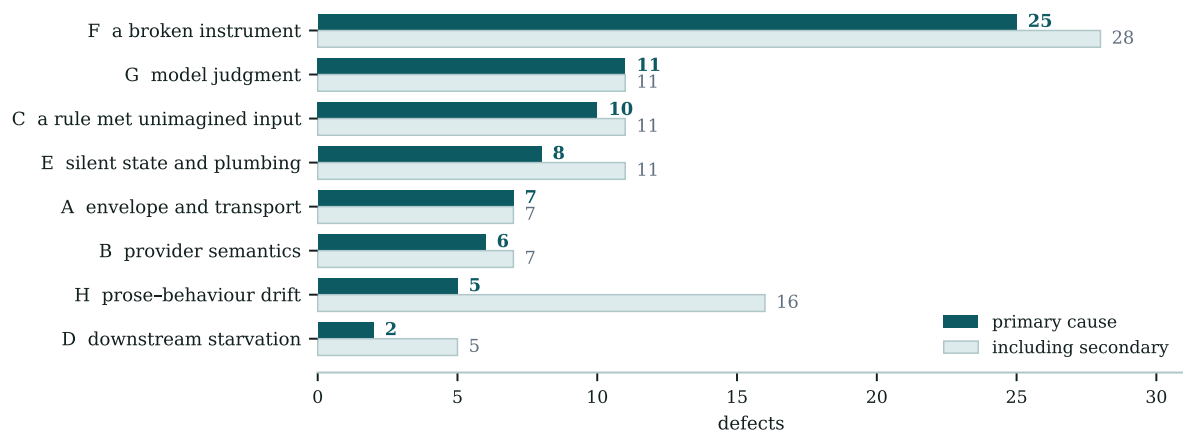


Figure 4: 74 defects across 110 sorted rows. The entry-by-entry assignment is in Appendix A.

The instrument was wrong more than twice as often as the model. I built this system to guard against a model being wrong. That guard largely held, and the failures moved to the thing doing the measuring.

All eleven judgment failures have one shape. The prompt said what I wanted without saying how the thing works. The first agent was given a field that can delete a restriction the user typed, and no rule about which side of a conflict may be removed, so it removed the restriction. The clearest case is the research agent writing searches like “*NextEra Energy renewable portfolio*”

expansion contracts”; five such searches returned one article between them, against a provider that requires every word to appear. The prompt’s own examples were right and named no company, but its heading read THE ARTICLES MUST NAME COMPANIES. The rule was about the kind of *event* to search for, and the model read it as an instruction to name the firm — a fair reading that nothing contradicted, because the prompt never said what the search actually does. Saying so fixed it: every extra word is another word the article must also contain, and unusual words cost the most. Before, one article and no candidates. After, fourteen articles, five themes, all on topic.

Writing that does not match behaviour is rarely the cause and often the cover. Five defects as a direct cause, sixteen once indirect ones are counted, by far the widest gap in the chart. One review of a single session’s work found seven defects, four of them a promise made in writing that the code did not keep: a note saying interruption was safe while the program printed an error trace, a comment saying the reader was given something to check while the web address was being broken across lines and could not be copied, and two entries claiming things worked that did not. Every one of those sentences was written in the same session as the code, by someone who had just finished convincing himself.

5.3 Two cases

A profit margin of 168. The final agent wrote “operating margin falls below 150” into a write-up. A margin cannot go above 1.0, so the model looked wrong. It was not: the data provider reported 168.38, and 150 is a sensible threshold for that number. The figure had travelled through two agents I had already checked. The ranking treats anything above 0.40 as perfect, so rubbish and excellence scored the same and the company took full marks on a figure that did not exist; with it rejected, its score fell from 0.373 to 0.060. The risk rules only look for negative margins and said nothing. The evaluation had a sanity limit for growth and none for margins. A person reading a sentence caught it, and the general lesson is that **a number that is capped cannot also be its own alarm** — everything above the cap looks identical, so the alarm has to sit before the cap.

Two months later the same group produced a subtler version: a gross margin of -44.9% next to an operating margin of +27.1%. Operating margin is gross margin minus costs, and costs are never negative, so on one set of accounts the second cannot be larger than the first. Both values passed the check written after the 168 case, because that check tests each number alone and neither is absurd on its own. It takes the pair, and nothing was looking at pairs. The bad number was not merely surviving — it was scoring well and was the reason that company ranked first.

The agent that was never given anything to cite. For several sessions I recorded that the final agent barely cited articles: one of eight conditions pointed at one, the rest generic thresholds that read the same for any company. I read this as the model taking the easy option. Which company produced that single citation is the clue. Of three candidates, one had no risks and so no articles reached the final stage; one had a single risk pointing at a single article, which produced the one cited condition; the third had two risks worked out from figures, which point at nothing. The object the final agent reads keeps only articles that a *risk* cited. **The model complied every time it could: one out of one, not one out of eight.** The evidence had been carried the whole way in a different field, on an object the final stage was never handed.

The fix was plumbing, and it was measured against an untouched run on the same saved state, so the only thing that changed was whether the research findings were passed in:

recorded run, months of argument about it	1 of 8 conditions cited
same inputs, research WITHHELD, re-run now	1 of 7
same inputs, research SUPPLIED	3 of 7

The middle row is what makes the result readable, because re-running the old behaviour in the same session reproduces the old number, so the jump is the fix and not the weather. The better half of the outcome is what got pushed out: one company went from three conditions to two, and the three it lost were “revenue growth turns negative”, “operating margin turns negative” and “gross margin falls below 0.20”. Giving the model something real to cite did not just add a citation. It displaced boilerplate.

5.4 A third case, whose subject is this paper

The first version of the chart in 5.2 was wrong in six of its eight rows. I produced it by adding up my own sorting from memory instead of counting the table I had just written, and I stated a total of 75 above a table that summed to 74. Every individual assignment was sound; the summary of them was not. It was not the only one. While drafting this paper I also claimed that a stale test count had drifted over five sessions, when in fact the two numbers were written into one file by one person a day apart and disagreed immediately, and I claimed the log is written in the first person when across thirty thousand words it uses “I” twice. Three claims about text sitting on a disk, all checkable in seconds, all wrong, each corrected by someone counting rather than by anyone thinking harder.

6. What is still wrong

Every item below was measured rather than guessed at, and each says what would settle it and why that was not built.



one real run on a renewable-energy profile — the drop from 9 to 3 is one article per theme, not a retrieval failure

Figure 5: One real run, end to end. Most of the loss happens in the first step.

The number of companies examined is capped by the number of themes, not by how much news is fetched. The company agent grades company-and-theme pairs, and the object it reads keeps only articles a theme cited. With a cap of five themes and most themes citing one article, the pool reaching that stage is capped near five however much news was fetched. I recorded this for two sessions as a shortage of articles, and it is not: fetching more was tried, the model merged five themes into three, and the result was worse on every axis. Raising the cap is a design decision about what “the research found” means to every later stage, not an

edit. In a well-covered sector it costs nothing. In a thin one it empties the write-up, which is why the problem is invisible in exactly the runs that go well.

The research agent records almost no disagreement. None of five test profiles produced a single dissenting view. No prompt fixes this, because most themes cite one article and one article cannot disagree with itself. Underneath is a free news tier that returns three articles per request. It is worked around rather than solved: the risk critic does its own opposing search, so disagreement enters one stage later than it should. That workaround is load-bearing. If those searches ever stop returning anything, the problem comes back with nothing behind it.

The source filter cannot cover its long tail. Across 272 articles from 130 sources, 86 of those sources contributed exactly one article each, and a list of publisher names cannot cover a spread whose most common case is one. Three alternatives were tried and rejected on real data. Filtering by website was killed fastest: six of seven articles from the worst offender are company announcements, and the seventh is a patent lawsuit, which is the one thing that must never be removed. What carries the weight instead is a filter on the *shape* of an article, with an override protecting judgemental and market-reaction language, so a company reporting bad results is never mistaken for a company announcing itself.

Two scoring limits, measured and accepted. The ranking tops out, so exceptional companies cannot be told apart, and financial companies cap at half marks because banks have no cost of goods. Both distort the absolute score and neither distorts an order anyone sees, which is why the score was removed from the reader's write-up: a number valid only for ordering invites a reader to take it for a percentage of something.

Off-topic articles reach the pile but not the output. In eight of nine recorded cases every theme the model produced was on topic. The cost is wasted fetching, not corrupted output. I described this wrongly twice in earlier entries as a problem in the output, and the correction matters because those entries would have justified spending time on it.

One of four exclusion reasons has never fired. Section 4 gives the reason it stays that way. It is also rarely reached, and that is itself a finding: the research agent respects restrictions when it searches, so a profile excluding semiconductors produces searches about software instead, and nothing disqualifying survives to be excluded. That check is a last line of defence which normal use walks around.

The twelve hard cases for the first agent stopped being readable. They hit 12 out of 12, and the guide written alongside them says 8 to 10 is a good hard set and 12 means it has become too easy: fixing the defect the set was built to catch used up its headroom. Three days later the same twelve scored 8, with every clarification case failing and two of those failing on repeat disagreement rather than on a wrong answer. Neither number tells you much about the agent. The first says the set has run out of range, the second that it has become unstable.

A failed run cannot be resumed. When the last stage fails the system records the error and shuts down cleanly, so a reader never sees a crash, and listing now reports the run as failed rather than finished. But the four stages already paid for cannot be continued. Making them resumable changes what resuming means for every run, so it is left as a decision rather than a task.

Nothing here is unknown. Three of the four things rejected in the closing sessions were killed by evidence gathered after the idea already looked good, and the fourth by what the evidence would have been worth. The instrument that decided every one of them was the same: hold something still and look again.

7. Limits of this evidence

One system, one model, one provider. Everything here came from a single pipeline running one open-weights model through one provider, with news and company data from three others, all on free tiers. The model was never compared against an alternative. The central ratio is a property of this model as much as of this architecture: a weaker model would push judgment failures up, a stronger one down, and neither tells you the ratio holds elsewhere.

The free tier shaped what could be measured. Six or seven full runs a day meant several findings were reached by replaying saved state because a live run was unaffordable, and some questions were left open because answering them cost more than the answer was worth.

The sorting was done afterwards, by one person. Defects were not labelled as they happened. I read the 95 entries I had written myself and sorted them into groups I had also defined, with nobody checking the assignments, so the group boundaries are not independent of the person who wrote the defects up. Two reclassifications made while drafting show how movable they are. Appendix A exists so a reader can argue with individual assignments, not only the totals.

The log records defects I noticed. It cannot record the ones no instrument caught, and that cuts in a particular direction: if the instruments were wrong 25 times, the most likely thing still hiding is another broken instrument, because that is the group whose failures are hardest to see from the output. That count should be read as a floor.

Appendix A --- how every entry was sorted

Entry number to main group, secondary after a dot. Letters mark entries holding more than one defect; – means no defect. Full text of every entry is in the log.

1 G·F	2 C	3 E	4a C	4b B	4c E	5 C·B	6 A	7 F	8 B	9 –	10 F
11 B	12 C	13 F·H	14 –	15 –	16 –	17 –	18a B	18b F	19a A	19b A	19c A·E
20 G	21 F	22 –	23 –	24a A	24b F·C	24c B·F	24d G·D	25 –	26 –	27 E	28 –
29 –	30 F	31 –	32 –	33 –	34 F	35 F	36 –	37 –·H	38 F	39 F·H	40 –
41 F	42a E·H	42b F	43 –	44 –	45 –	46 F·H	47 –·H	48a C·H	48b C	48c E	49a H
49b H	49c F	49d F·H	50 E	51 –	52 –	53 C	54 C	55 G	56 F·H	57 F	58 F
59 F	60 G	61 G	62 D	63 H	64 C·E	65 G	66 F	67 F	68 G	69 –	70 –·D
71 –	72 –	73 G	74 –	75 D	76 –	77a G	77b A	78 B	79 –	80 –	81 E
82 A·E	83 –	84 C	85 G	86 –	87 F·D	88 E·H	89 –	90 F	91 –	92 H	93 –
94 H·F	95 F·H										
TOTALS	110 sorted rows, 74 defects										
	F 25	G 11	C 10	E 8	A 7	B 6	H 5	D 2			